

Running on Amarel Cluster - Quick Guide

This guide shows you how to run experiments on the Amarel HPC cluster at Rutgers.

Prerequisites

- Your Rutgers NetID (e.g., **abc123**)
 - SSH access to Amarel
 - A conda environment set up for your project
-

1. Logging In

From your local computer:

```
ssh NETID@amarel.rutgers.edu
```

Replace **NETID** with your actual Rutgers NetID.

2. Navigate to Project and Setup Environment

Once logged into Amarel:

```
# Navigate to your project directory
cd /home/NETID/PROJECT_DIR

# Load conda (if not already loaded)
source ~/miniconda3/etc/profile.d/conda.sh

# Activate your project environment
conda activate ENV_NAME
```

Replace:

- **PROJECT_DIR** with your actual project directory name
- **ENV_NAME** with your conda environment name

Important: Set up temporary directory

Amarel's **/tmp** directory is shared and can cause permission errors. Use your home directory instead:

```
mkdir -p /common/home/NETID/tmp
export TMPDIR=/common/home/NETID/tmp
```

To make this permanent, add the `export` line to your `~/.bashrc`:

```
echo 'export TMPDIR=/common/home/NETID/tmp' >> ~/.bashrc
```

3. Quick Test (Login Node Only)

On the login node, you can only run very short tests. For example:

```
cd /home/NETID/PROJECT_DIR
conda activate ENV_NAME
export TMPDIR=/common/home/NETID/tmp

# Test that your environment works
python your_script.py --help
```

Important: Don't run long training jobs on the login node. Use compute nodes instead (see below).

4. Interactive GPU Session (For Debugging)

For debugging or testing, you can request an interactive GPU session:

```
salloc --partition=gpu --gres=gpu:1 --time=02:00:00 --mem=32G --
constraint=ampere
```

Once you get a compute node, run your code:

```
cd /home/NETID/PROJECT_DIR
conda activate ENV_NAME
export TMPDIR=/common/home/NETID/tmp

# Example: run a short test
python your_script.py \
  --arg1 value1 \
  --arg2 value2
```

When done, type `exit` to release the node.

5. Submit Batch Jobs (Recommended for Real Runs)

Create a Slurm batch script. Example: `run_job.sbatch`

```
#!/bin/bash
#SBATCH --job-name=your_job_name
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --time=24:00:00
#SBATCH --mem=64G
#SBATCH --constraint=ampere
#SBATCH --output=logs/%x_%j.out
#SBATCH --error=logs/%x_%j.err

source ~/miniconda3/etc/profile.d/conda.sh
conda activate ENV_NAME

export TMPDIR=/common/home/NETID/tmp

cd /home/NETID/PROJECT_DIR

python your_script.py \
    --arg1 value1 \
    --arg2 value2
```

Submit the job:

```
cd /home/NETID/PROJECT_DIR
sbatch run_job.sbatch
```

6. Check Job Status

See your running/pending jobs:

```
squeue -u NETID
```

See job history:

```
sacct -u NETID --format=JobID,JobName,State,Elapsed
```

Check logs after job completes:

```
cd /home/NETID/PROJECT_DIR/logs
ls -lt
less your_job_name_<JOBID>.out
less your_job_name_<JOBID>.err
```

Check output directories:

```
cd /home/NETID/PROJECT_DIR/outputs  
ls
```

7. Common Slurm Options

Here are useful options for your batch scripts:

- `--partition=gpu`: Use GPU partition
- `--gres=gpu:1`: Request 1 GPU
- `--constraint=ampere`: Request A100 GPUs (best performance)
- `--time=24:00:00`: Maximum runtime (24 hours)
- `--mem=64G`: Request 64GB RAM
- `--output=logs/%j.out`: Where to save stdout
- `--error=logs/%j.err`: Where to save stderr

8. Cancelling Jobs

To cancel a running job:

```
sancel JOBID
```

To cancel all your jobs:

```
sancel -u NETID
```

Troubleshooting

Permission denied errors with `/tmp`:

- Make sure you set `export TMPDIR=/common/home/NETID/tmp` before running your code

Job gets killed:

- Check if you exceeded time limit (`--time` in your script)
- Check if you exceeded memory limit (`--mem` in your script)
- Look at the `.err` log file for error messages

Can't find conda:

- Make sure you run: `source ~/miniconda3/etc/profile.d/conda.sh`
- Or check where your conda is installed: `which conda`

Quick Reference

Replace placeholders in all commands above:

- **NETID**: Your actual Rutgers NetID
- **PROJECT_DIR**: Your project directory name
- **ENV_NAME**: Your conda environment name

Common paths (customize as needed):

- Project directory: `/home/NETID/PROJECT_DIR`
- Temporary files: `/common/home/NETID/tmp`
- Logs: `/home/NETID/PROJECT_DIR/logs`
- Outputs: `/home/NETID/PROJECT_DIR/outputs`